

Lemma Litepaper

Version: v0.1 draft **Date:** May 2026 **Repository:** Lemma **Status:** Draft for review

Abstract

Lemma is a Bittensor subnet designed to produce open, verifier-grounded training data. Its first production domain is Lean theorem proving: miners search for formal proofs, validators check submitted artifacts with a deterministic pinned Lean environment, and accepted proofs become replayable public corpus rows.

The central thesis is simple: modern reasoning models need more than plausible text. They need checked artifacts that can be replayed, audited, retrieved, fine-tuned on, and used in repair loops. Lemma turns deterministic verification into a market mechanism for producing those artifacts.

In v1, Lemma is intentionally narrow. It is not a generic code benchmark, a subjective reasoning contest, a smart-contract escrow product, or a Google DeepMind Formal Conjectures payout path. It is a Lean proof-data engine. Math is the wedge; verified data is the product; the graph-shaped corpus is the substrate; the market is the means.

1. The Problem: AI Can Guess, Verifiers Can Check

Large language models have become increasingly capable at mathematical reasoning, code generation, and formal problem solving. But most model outputs remain probabilistic text. They may be useful, elegant, or convincing while still being wrong. For high-stakes reasoning, plausibility is not enough.

Formal verification offers a different kind of signal. A Lean proof either passes the checker in a specific environment or it does not. That binary signal is valuable because it can transform model-generated reasoning into an auditable artifact. A verified theorem-proof pair can be replayed, indexed, deduplicated, attributed, and used as supervised training data.

The bottleneck is not merely evaluation. The field needs a continuous source of checked reasoning artifacts: not just benchmark scores, but open rows that contain enough metadata for future operators to reconstruct the task, replay the verifier, and train better systems. Lemma exists to create that source.

2. What Lemma Is

Lemma is a permissionless proof-data subnet. It coordinates three roles:

- 1 **Task supply:** a deterministic active set of formal tasks.
- 2 **Miners:** participants that search for valid artifacts, beginning with Lean proofs.
- 3 **Validators:** participants that run pinned deterministic verifiers, score accepted artifacts, and publish corpus rows.

The Lemma loop is:

```
formal task -> artifact search -> deterministic verification -> proof-unit score -> public corpus row
```

For v1, the artifact is a Lean proof and the verifier is a pinned Lean/mathlib environment. Miners may use any method to discover proofs: local tactics, retrieval, hosted models, custom agents, human-written proofs, or hybrid systems. Validators do not score effort, model identity, prose quality, or claimed reasoning process. They score only the final artifact.

This makes Lemma a data-production mechanism rather than a subjective competition. Its output is not a leaderboard alone; it is a growing corpus of checked artifacts designed for model training, retrieval, repair, and evaluation.

3. What Lemma Is Not

Clear boundaries matter. Lemma v1 is not:

- a general natural-language reasoning benchmark
- a generic code-task subnet
- a prose-judging system
- a smart-contract escrow product
- an owner-cut routing mechanism
- a contract-custody system
- a Google DeepMind Formal Conjectures payout path
- endorsed by Google DeepMind
- a production multi-domain verifier market yet

Google DeepMind Formal Conjectures, miniF2F, PutnamBench, lean-eval, and related benchmarks are useful downstream measurement surfaces. They are not the v1 payout path. If models trained on Lemma's corpus later solve more frontier benchmark problems, Lemma is working. But the subnet itself pays for verified production rows, not benchmark branding.

4. Why Lean First

Lean is the first production domain because it offers a mature deterministic verifier and a high-value artifact type: theorem-proof pairs. A Lean proof can be checked mechanically in a pinned environment. This gives Lemma a clean correctness boundary.

Lean also sits at a strategic point in the development of reasoning AI. It connects mathematical problem solving, symbolic reasoning, program verification, proof search, retrieval, curriculum learning, and model-assisted repair. A large open corpus of verified Lean rows can support:

- supervised fine-tuning of theorem provers
- retrieval systems for proof search
- proof repair and self-correction loops
- reinforcement learning from verifier feedback
- evaluation of formal reasoning models
- transfer into code reasoning and program verification

Lemma starts with math because math provides a clean wedge. The broader product is verified data.

5. Mechanism Overview

Each validator epoch contains K active paid task slots. A miner earns credit when it is the first miner in that epoch to submit a unique artifact that passes the deterministic verifier for a task.

The v1 scoring rule is:

```
credit(miner) = count(first_valid_unique_verified_artifact_per_task_by_miner)
score(miner) = credit(miner) / K
weight(miner) = credit(miner) / K
unearned_share = 1 - sum(miner_weights)
```

This creates a fixed-denominator system. If only some tasks are solved, only those solved slots pay current miners. The unsolved portion is not redistributed to current solvers by default. Instead, the default policy is burn, with explicit future policy rails for recycle or hold.

This matters because redistribution can overpay easy tasks and reduce pressure on frontier proof production. Lemma's default design preserves the signal that unsolved work remains unsolved.

6. Miner Workflow

A miner's job is to produce a valid artifact for an active task. For Lean v1, that means producing a proof script bound to the task identity, task version, and target hash.

A miner may use any search stack:

- handcrafted Lean proofs
- tactic search
- retrieval from prior corpora
- local language models
- hosted model APIs
- tree search and repair loops
- hybrid human-in-the-loop systems
- specialized proof agents

Lemma intentionally keeps provider and model logic on the miner side. The subnet should not care whether a proof came from a frontier model, a small local model, an automated tactic, or a human. If the submitted artifact passes the pinned verifier and satisfies protocol rules, it can earn credit.

The miner interface is therefore artifact-oriented. A miner receives or fetches active tasks, attempts proof search, packages a submission, signs or binds it as required, and submits it for validator checking.

7. Validator Workflow

Validators maintain the active task set, receive task-bound submissions, run verifier adapters, compute weights, and write accepted rows to the corpus.

A valid validator path includes:

- 1 loading the pinned task registry;
- 2 reconstructing the active task set deterministically;
- 3 validating submission metadata;
- 4 dispatching the artifact to the domain verifier;
- 5 enforcing security and anti-gaming rules;
- 6 deduplicating accepted artifacts;
- 7 assigning proof-unit credit;
- 8 computing miner weights as credit / K ;
- 9 accounting for unsolved-slot value;
- 10 writing replayable corpus rows.

The validator does not need to understand the miner's search process. It only needs to reproduce the task, run the pinned verifier, and determine whether the artifact passes.

8. Corpus as the Product

The Lemma Corpus is the main product of the subnet. Each accepted row is designed to be replayable, attributable, and useful for model training.

A row can include:

- schema version
- row ID
- task ID and task version
- theorem name
- theorem statement and type expression
- imports
- Lean toolchain and mathlib revision
- target hash
- proof script
- proof hash
- proof identity and identity strength
- source stream and source license
- solver hotkey
- validator hotkey
- reward status
- dependency metadata
- graph links
- verification summary

Failed proofs are not corpus rows. Valid alternate proofs can be stored with `rewarded: false`, allowing the corpus to preserve useful diversity without paying duplicate credit for the same slot.

The value of a row depends on replayability. Another operator should be able to reconstruct the task, load the pinned environment, and rerun the verifier. Without replay metadata, a row is not production-quality verified data.

9. Graph-Shaped Data

Lemma rows are graph-shaped from the start. Each accepted artifact links task, proof, proof identity, source, verifier, solver, validator, dependencies, and verification metadata.

This graph structure is not cosmetic. It enables future systems to ask richer questions:

- Which proofs solve related theorem families?
- Which imports and dependencies are common among solved tasks?
- Which sources produce high-value verified rows?
- Which proof identities are duplicates or near-duplicates?
- Which validators accepted which rows under which verifier versions?
- Which corpus regions are useful for downstream training or retrieval?

The graph is the substrate shape. Lean is the first production domain, but the row contract is designed so future deterministic verifier domains can attach to the same substrate.

10. Proof Identity and Deduplication

A proof market needs deduplication. Without it, miners could resubmit the same solution repeatedly or make superficial script changes that do not represent distinct verified artifacts.

Lemma's preferred identity is a strong proof-term identity when available. During early Lean operation, the system may fall back to script-based hashes such as normalized script SHA256 or script SHA256. These fallbacks are explicitly labelled as weak identity.

The distinction matters. Weak script identity is useful for early operation, but it is not a full structural proof identity. In production mode, full reward requires strong proof identity. This pushes the system toward better canonicalization and stronger deduplication as the corpus matures.

11. Security and Anti-Gaming Boundaries

Lean provides the core binary correctness signal, but a subnet also needs clear adversarial boundaries. Validators reject artifacts that violate task, verifier, or corpus integrity.

Examples of rejected behavior include:

- `sorry` or `admit`
- custom axioms or constants
- unsafe code or native execution tricks
- changed theorem statements

- disallowed imports
- macro, syntax, elaborator, or notation changes
- oversized proof bodies
- inactive task IDs
- mismatched task versions or target hashes
- unsigned live miner responses

Production verification should run in a pinned Lean/mathlib environment with networking disabled. Registry bytes should be pinned, and production registry signature checks should fail closed.

The goal is not merely to prevent invalid proofs. It is to ensure that accepted rows remain useful public artifacts rather than contaminated data.

12. Architecture

Lemma is organized around a domain-neutral verifier spine with Lean as the only active production domain.

The implemented architecture includes:

- task schema, provenance, registry loading, and target hashing
- deterministic task queues and activation gates
- task-bound submission schemas
- miner-side local command adapters
- validator-side submission validation, verifier dispatch, scoring, and corpus writing
- first-valid-unique scoring
- verifier adapter interfaces and a Lean adapter
- corpus validation, replay, indexing, and export helpers
- graph nodes and dependency edges
- typed future interfaces for commitments, randomness, weights, and burn/recycle rails

Two controllers shape the active task frontier:

- **frontier_depth**: a difficulty proxy driven by solve-rate behavior
- **active_K**: a throughput target driven by validator capacity

A low or zero solve rate should halt frontier advancement and request hard-target variants rather than pushing the queue backward into already exposed tasks.

13. Domain Adapter Model

Lemma is designed to generalize beyond Lean, but not prematurely. A future production domain must satisfy a strict contract:

- deterministic verifier
- pinned verifier version
- task schema
- submission schema
- normalized corpus row format
- sandboxing rules
- timeout and memory limits
- duplicate policy
- public license
- scoring function
- adversarial tests

A future domain cannot enter production rewards merely because it is interesting. It must provide the same guarantees that make Lean useful: deterministic checking, replayability, provenance, licensing, identity, and corpus value.

Potential future domains include:

- Verus for Rust programs, specifications, and proofs
- SAT/SMT formulas with assignments, traces, or certificates
- LP/SDP optimization instances with primal/dual certificates
- cryptanalysis instances with verifiable witnesses

These are roadmap directions, not live production mechanisms.

14. Economic Design

Lemma uses normal Bittensor miner and validator emissions. It does not custody funds, route owner emissions through contracts, or introduce a smart-contract escrow layer.

The economic design is centered on verified proof units. Each active task slot contributes one unit of possible credit. If a miner is first to submit a unique passing artifact for a task, that miner earns the unit for the epoch. If no one solves a task, the slot remains unearned.

The unearned-share policy is important. By default, unearned share is burned rather than redistributed to successful miners. This keeps the denominator honest and prevents a small number of solved tasks from absorbing the value of the unsolved frontier.

Future recycle or hold policies can route value toward additional proof-production rails, but such policies should be explicit. The default is conservative: pay verified production, do not pay unsolved work, and do not inflate easy wins.

15. Data Consumers

Lemma's immediate participants are miners and validators, but its downstream customers are model builders.

The corpus is intended for:

- theorem-proving models
- formal-reasoning agents
- retrieval-augmented proof search
- proof-repair systems
- code-reasoning and program-verification models
- benchmark and evaluation pipelines
- other Bittensor subnets that consume verified reasoning data

Affine-style model miners can consume Lemma's public corpora, but Lemma does not require a transactional dependency on Affine or any specific model competition layer. Lemma is the verifier-grounded data production layer.

16. Roadmap

Phase 1: Lean Corpus Engine

Stabilize the Lean verifier path, improve miner and validator reliability, preserve binary accepted/rejected scoring, and export replayable Lean-domain corpus rows.

Phase 2: Corpus Productization

Add dataset cards, JSONL and Parquet exports, Hugging Face-style releases, train/validation/test splits, exact duplicate detection, and near-duplicate metadata.

Phase 3: Verifier Adapter Architecture

Generalize the adapter interface, domain registry, schema v2, and runtime metadata for pinned verifier containers.

Phase 4: Verus Pilot

Introduce an experimental Verus adapter for Rust specifications and proofs, including sandboxing requirements and sample fixtures.

Phase 5: Multi-Domain Lemma

Expand to additional deterministic verifier domains such as SAT/SMT certificates, optimization certificates, cryptanalysis witnesses, and other domains with clear corpus value.

17. Risks and Open Questions

Weak proof identity during early operation

Script hashes are useful but imperfect. Production-grade deduplication requires stronger proof-term identity.

Task quality and licensing

A verified row is only useful if the task source is legitimate, licensed, and valuable. Lemma must avoid unknown, restricted, or benchmark-contaminated sources in paid activation.

Solver overfitting

If active tasks are too predictable, miners may optimize for narrow patterns rather than general proof capability. Curriculum and frontier controls must preserve useful pressure.

Validator reproducibility

Validators must agree on task registries, verifier versions, and active task sets. Any nondeterminism can damage scoring integrity.

Corpus usefulness

A large number of trivial proofs is less valuable than a smaller number of diverse, replayable, well-metadated rows. Lemma must optimize for useful verified data, not just row count.

Multi-domain expansion

Adding domains too early could dilute the brand and weaken guarantees. The Lean wedge should work before Lemma becomes multi-domain.

18. Conclusion

Lemma converts formal verification into an open data-production market. It begins with Lean because Lean provides a deterministic correctness boundary and a high-value artifact type. Miners search for proofs, validators check them, scoring rewards verified unique artifacts, and accepted rows become a replayable public corpus.

The long-term opportunity is larger than theorem proving alone. Many future AI systems will need checked reasoning traces, verified programs, solver certificates, optimization witnesses, and proof-carrying artifacts. Lemma's first job is to make one domain work: Lean proof production as a reliable subnet. If that succeeds, the same graph-shaped substrate can support a broader market for verified intelligence.

AI can guess. Verifiers can check. Lemma pays for checked data.